Architectural Overview of Comma.ai

EECS 4314 - Advanced Software Engineering

York University

Authors

Group C

Student Name	Student Number	Email
Ricky Tran	218809269	ricky4@my.yorku.ca
Ricky Nguyen	219461201	ngurky@my.yorku.ca
Misheel Tuguldur	219874742	mish1926@my.yorku.ca
Andrew Nong	219661537	an54@my.yorku.ca
Daniel Yonattan	219594761	daniel74@my.yorku.ca

October 25, 2026

Abstract

This report presents a conceptual architectural analysis of comma.ai's OpenPIlots advanced driver assistance system using publicly available documentation and technical publication. The report also evaluates the systems architectural styles, design patterns, concurrency, communication mechanisms, external interfaces, and performance considerations. Use cases, sequence diagrams, state diagrams, and architectural diagrams are used to illustrate important data and control flows throughout the system. Overall, the analysis shows how OpenPilot uses a modular style, messages for communication, control feedback loop architecture to coordinate driver assistance functions.

Introduction and Overview

The purpose of this report is to give newcomers, developers, and project managers better insights into comma.ai, an open-source driver assistance tool that supports brand-renowned vehicles from Toyota, Honda, and Hyundai. Traditional, built-in driving systems are sometimes insufficient, high cost, or fragmented. comma.ai aims to address these problems by making advanced driver assistance universally available, optimizing lower cost cameras and processing hardware, and adding transparency to blackboxed vehicle software. This opens the gates for consumers to purchase standard vehicle models without losing the futures from brand-new luxury cars and allows drivers to take control of the technology in their own cars. Although the end-goal is clear, there are many open questions on how this solution will make its place in the market. The purpose of this report will be to evaluate Openpilot's architecture, software dependencies, and safety mechanisms to assess how an open-source, end-to-end system competes with proprietary Original Equipment Manufacturer Advanced Driver Assistance Systems (OEM ADAS).

The architecture section will explore Openpilot's global architectural styles, key subsystems and processes, performance critical constraints, and evolvability. Through this section, readers will be able to understand how multi-process architecture scales independent processes, manages communication between those processes using shared memory buffers, handles fault tolerance, and uses real-time sensor processing pipelines.

The dependency diagram section will give a visual overview of the underlying architecture, data, and control flows.

External interfaces, how system communicates with CAN bus via panda, user interface, cloud and telemetry interface understand primary use cases of a driver assistance system, the flow between each process

By the end of this report, individuals or teams should be able to collectively contribute to this tool and even challenge existing architectural solutions.

Architecture

High Level Architecture

Openpilot is an open-source, driver assistance software that is built to replace a car's built-in adaptive cruise and lane centering mechanisms with its own. It is typically hosted on a dedicated, wind-shield mounted hardware device called the comma four. When the device is powered on, launch_openpilot.sh runs launchchffrplus.sh to setup the environment (paths, updates, AGNOS on comma hardware) and runs build.py to compile the concurrent background services for manager.py to start and maintain. Openpilot is generally organized into many separate programs, each running at the same time to support all kinds of features, rather than existing as one big program. This architectural style is supported by a supervisor or management program called manager.py, which is not only responsible for starting the applications, but also for managing them depending on the car type, whether the car is onroad or offroad, and other settings and modes. It also ensures each program is healthy while the device is running. There are five core processes to be aware of: I/O sensors, perception states, platform, planning and control, and vehicle interface. Each of these processes carry out different tasks, but rely on each other for information and proper controls.

Starting with the I/O sensors, this process is meant to capture the physical world through cameras, the Inertial Measurement Unit (IMU), and Global Positioning System (GPS). The camerad and webcamerad programs are used to capture road conditions and the individual driver as photographic and video data. It uses VisionIPC, which is a data pipeline for passing video and picture information to requesting services. The IMU is supported by a program called sensord, which reads the accelerometer and gyroscope data. qcomgpsd, ubloxd, and pigeond are the three programs that support the GPS to determine the vehicle's exact location, speed, and position in an internal map. pandad sits between all of these programs as a translator box in the car. Virtually all modern passenger cars, light trucks, and SUVs use CAN (Controller Area Network) buses to allow internal components to communicate with each other. Openpilot, however, uses cereal, which is a named message format for daemons to communicate with each other and isn't compatible with CAN buses. To solve this problem, Panda connects Openpilot to the vehicle's CAN bus, while pandad and card receives, interprets, and publishes those CAN as cereal messages.

The perception and state is another process managed by manager.py, and is responsible for turning the sensor and video information received into actionable states. These states are used to determine what is around the car, where it is exactly, and whether or not the driver is paying attention to the road. Starting with the first program, modeld is responsible for pulling road frames from VisionIPC and outputs information such as how far the lead car is, which direction the lane is curving to, and more. locationd mixes information it receives from the IMU, GPS, and cameras to numerically determine the car's exact location and movement. paramsd is used to determine the car's state, such as its wheel angle, speed, tire wear, load, and more. calibrationd is used to align and straighten the car's cameras. torqued is used to tune steering torque behavior, and lagd is used to estimate the delay between inputs and actions. Lastly,

modeld is responsible for determining what the driver is doing, how they're posed, if they're attentive, and determining whether the system should alert the driver.

After the I/O sensors and perception state processes capture and normalize driver, car, and road conditions, planning and control processes take over to determine what speed or path to target, should Openpilot drive, and what steering, throttle, or brake to request. Much like the previous two processes, the planning and control also consists of many programs. selfdrived decides if Openpilot should engage or disengage assisted driving. radard picks which car ahead to follow, while plannerd and controlsd determines how fast the car should be going, whether or not to accelerate or stop, and by how much respectively. maneuversd and lateral_maneuversd are responsible for longitudinal and lateral tune testing to ensure plannerd and controlsd are working properly. The last two supporting daemons for this process are joystickd and joystick, which are responsible for debugging simulated manual drives.

Apart from main driver assistance processes, Openpilot also includes platform processes which are responsible for monitoring device health, user interface, logging and uploads, cloud, and streaming. Specifically, the hardwared daemon is used to monitor device health and onroad/offroad signals. The ui and soundd daemons are responsible for the on-screen user interface (UI) and alert sounds respectively. webrtcd and stream_encoderd are responsible for live video streaming and encoding to optimize storage spaces. loggerd, encoderd, and uploaderd are used to record system logs, compress videos to optimize storage spaces, and upload travel information to the cloud. athenad is also used to ensure connection to comma infrastructure and other tools.

The vehicle interface process is the last major process that connects all of them together. It is built to support the Panda and pandad daemons from the I/O sensor processors by decoding CAN messages into interpretable cereal messages through one program: card. This effectively allows the two systems, Openpilot and the car's native CAN bus, to gather information and execute commands.

Subsystems and Responsibilities

Openpilot is generally organized into many separate programs rather than existing as one big program. The collection of services work together and communicate through a messaging system [2]. For our conceptual architecture, we grouped these responsibilities into several major subsystems: Sensor Acquisition, Driving Model and Environment Understanding, Localization and Calibration, Driver Monitoring, System Supervision, Planning and Control, Vehicle and Safety, and Logging.

Starting with the Sensor Acquisition, this process is meant to capture the physical world through cameras, the Inertial Measurement Unit (IMU), and Global Positioning System (GPS). Comma.ai's published architecture stated that that *camerad* program is used to capture road conditions and the individual driver as photographic and video data. It uses VisionIPC, which is a data pipeline for passing video and picture information to requesting services [2]. The IMU is supported by a program called *sensord,* which reads the accelerometer and gyroscope data [2]. Positioning information is also available through GNSS. In comma.ai's documented architecture, *ubloxd* parses GNSS data for

use by localization services [2]. Together, these sources provide information about the road, driver, motion of the device, and position of the vehicle.

The Driving Model and Environment Understanding subsystem is responsible for converting road camera information into useful information about how the vehicle should move and what surrounds it. Comma.ai describes *modeld* as reading the road camera image stream from VisionIPC and running the main driving neural network. The resulting information includes the desired driving path and information about elements such as lane lines, lead vehicles, and road edges [2]. This allows raw camera information to be transformed to be used by planning and controlling portions of the system.

The Localization and Calibration subsystem determines how the vehicle and mounted device are moving and oriented. Comma.ai's architecture documentation describes locationd as combining noisy sensor measurements to estimate quantities such as position, orientation, velocity, angular velocity, and acceleration [2]. Due to the fact that comma devices can be mounted differently, *calibraitond* accounts for the device so camera information can be interpreted relative to the vehicle [2]. The documented *paramsd* service also estimates driving parameters like steering ratio, which are useful for accurate vehicle control [2].

Driver Monitoring performs a separate safety related responsibility. The driver facing camera is used by dmonitoringmodeld to estimate properties such as the drivers head pose and eye state [2]. The documented dmonitoringd service then uses this information to assess whether the driver is capable of taking control and to trigger alerts when necessary [2]. Current OpenPilot safety documentation also states that the driver must remain alert while OpenPilto is engaged and that the Driver Monitoring feature alerts the driver when distraction is detected [4]. More recent comma.ai documentation describes selfdrived taking in Driver Monitoring state to handle system states and alerts [10].

After sensor information is interpreted, Planning and Control determines how the vehicle should move. In the referenced architecture by comma.ai, planning determines smooth lateral and longitude behaviour using information such as the desired driving path and surrounding vehicles [2]. Control converts these desired steering and acceleration into commands that can be applied to the vehicle through a closed loop control system [2].

On the other hand, OpenPilot also includes System Supervision, Logging, and other supporting services. Comma.ai includes a management service responsible for starting and stopping OpenPilot processes and a user interface responsible for presenting system and driving information to the driver like inattention alerts [2]. OpenPilot also records driving sessions. Logging documentation states that routes are divided into one minute segments and that OpenPIlot records messages passed among its processes along with encoded camera streams [8]. Overall, this system support communication with the driver, monitoring operations, and debugging/development.

The Vehicle Interface and Safety subsystem connects OpenPilots internal driving decisions to the physical vehicle. OpenPilot documentation explains that each

supported vehicle must be individually ported because vehicle APIs and capabilities differ between makes and models [6]. Openpilot, however, uses cereal, which is a named message format for daemons to communicate with each other and is not compatible with CAN buses. So, the vehicle interface translates from vehicle state and control commands (OpenPilot) to the vehicle's specific controller area network (CAN) communication. Comma.ai states that *card* is the service that receives raw CAN stream from the vehicle and builds CAN messages used to control the car [9].

Panda provides an extra safety net between OpenPilot and the vehicle. OpenPilots glossary defines Panda as the secondary processor that implements functional safety and communications directly with the vehicle over CAN [7]. Ultimately, the vehicle interface and Panda allow OpenPilot to receive information from the vehicle while providing safety checks and controls.

Architecture Styles

Based on these responsibilities and information flow, OpenPilot can conceptually be understood as a mixture of architecture styles combining pipe and filter, process control, and publish & subscribe. The general data flow is starting from the Sensor Acquisition collecting data from the sensors, while the Perception and State subsystem transforms the data into estimates of the road, driver, and vehicle. Planning then uses the data to determine desired vehicle behavior, and control calculates the commands required to achieve it. The flow of data through these components can be reflected by the pipe and filter architecture via transforming the data between each processing stage. The entire system would also be utilizing the process control style more specifically the feedback control system. Since commands affect the vehicle’s motion that is measured which can therefore be measured again. Subsequent planning and control decisions can respond to the changes in vehicle behavior. The system supervisor can then govern the status of driving assistance while services support through monitoring, logging, and driver communications. Publish and Subscribe can be used through publishing information through named channels of which the “subscriber” can receive the data for use in multiple scenarios such as sending data between, planning, control, driver monitoring, and logging simultaneously. By using the publish and subscribe architecture the system can effectively stay loosely coupled and each individual component only receives what it is subscribed to. Data repository can be used in this application as a one authoritative truth of data, but within a real time system with lots of estimated or changing data writing to one main repository of data is slow and unnecessary as each individual subsystem requires different types of data. Taking this in mind the controls run at 100Hz [2] to actively assist with driving, updates of data would have to be faster than every 10 milliseconds for each control command.

Design Patterns

One design pattern that was observed when studying OpenPilot’s documented architecture is the Adapter pattern. OpenPilot supports vehicles of different brands and models. Each of these varying vehicles present different CAN messages, interfaces, and controls. OpenPilot’s car port documentation explains that each supported vehicle must therefore be individually ported to work with OpenPilot [6]. The vehicle interface

acts as an adapter between OpenPilot's generic driving logic and the communication format expected by the vehicle. The *card* process receives a raw CAN stream from the vehicle and constructs CAN messages to control the vehicle [9].

This resembles the Adapter pattern because it allows systems with incompatible forms to work together through a common representation. As a result, OpenPilot does not need to understand the unique CAN communication of every supported vehicle.

Concurrent Processes

OpenPilot is concurrent by design because its separate services operate independently while exchanging information through its publish-subscribe messaging system [2]. For example, Driver Monitoring can evaluate the driver while Driver Model processes road information and Planning and Control updates vehicle behaviour. These activities do not need to finish one after another.

System Communication and Performance

The eight processes might look like the whole picture, but Openpilot's software is actually far more complex. Satisfying customers and stakeholders with elaborate features is not the only challenge to creating a driving assistance system. The real challenge is making sure those processes and features are communicating properly while the vehicle is on the road. To tackle this challenge, Openpilot uses msgq, a shared-memory publishing and subscription streaming pipeline. Cereal sits on top of this service, using Cap'n Proto Events to send serialized data as msgq messages for channels to publish and subscribers to receive. While this approach works well for numerical data, pictures often have a harder time being transported because of how large they naturally are. To address this problem, Openpilot uses VisionIPC, a library in the msgq submodule for passing camera frames through processes. Together, these two systems move live data throughout the entire service to ensure up-to-date data is being served to core features for optimal driver assistance.

Another real challenge is designing the system to be scalable and testable. Each new feature shouldn't demand dangerous tests on the road to confirm whether or not they are working or not. This is exactly why Openpilot opted to design a driving stack consisting of a core manager program and several daemons. When new features are in demand, a new daemon for that feature can simply be created and added to the *process_config.py* list to be managed by *manager.py*. This modular approach also makes testing much easier. Instead of risking full test-drives, there are isolated unit tests and regression tests for individual classes, functions, or even whole end-to-end flows. The heaviest and most important tests are the regression tests, or process replay tests. These tests use real, recorded logs from route segments and compares those logs to continuous integration runs triggered by pull requests to determine whether or not a new feature would break a working process or daemon.

The last challenge is meeting performance demands. After all, what point would there be if the system was even slower than the driver? Each daemon has a specific target rate on how long it can take to handle a specific job. For example, the *modeld* daemon

has a target rate of 50 milliseconds per step or 20 Hertz. If it cannot process a message within those limits, then it will simply drop that stale data and move onto the next message, otherwise, the whole system could quickly become outdated. Similarly, the *controlsd* daemon is limited to 100 Hertz (10 milliseconds per step) between messages to minimize any lag between steering or throttling and keep driving smooth between vision updates. These loop target rates ensure data freshness in the entire system, minimizes lag, and ensures something like a lane change that should have happened five minutes ago is not happening now.

Overall, Openpilot's modular architecture, high-speed message pipelines, and strict performance rates demonstrates what a robust, user-ready system looks like. Beyond the core design choices highlighted in this section, Openpilot features many more low-level architecture decisions that showcases just how complex a driving assistance system can be without becoming unnecessarily complicated.

Architecture Diagram

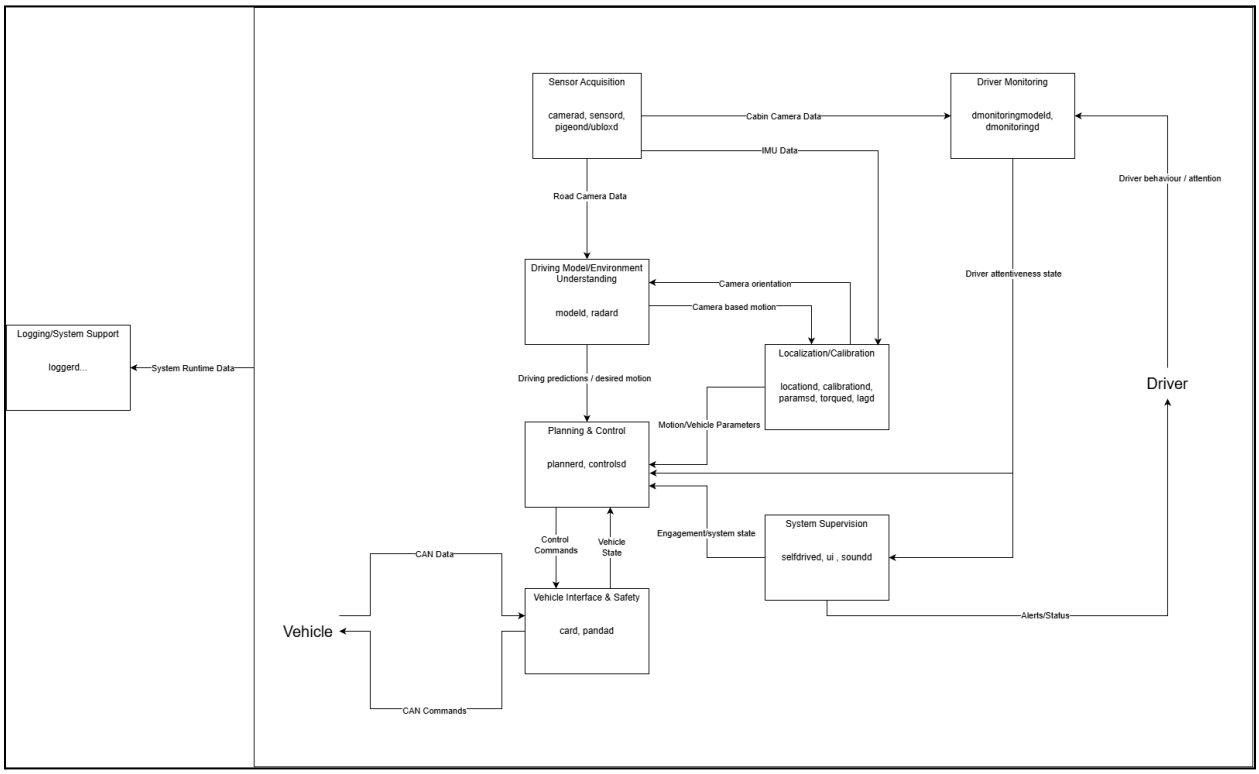


Figure 1. Group conceptual architecture of OpenPilot based on publicly available documentation

External Interfaces

The primary external interface that Comma AI's system interacts with is the vehicle's CAN bus. The CAN bus is the vehicle's network protocol that allows the vehicle's ECUs to exchange information without requiring a separate wire per connection. Comma AI's

system is designed to work with a wide variety of vehicle makes/models, this presents several hurdles with respect to device communication. For starters, while some CAN bus communication is standardized, much of it is not. Additionally, different models of vehicle have different capabilities. In order to overcome these hurdles, Comma AI defines its own CarState, CarControl, and CarParams structures that represent the cars current state, control commands that openpilot wants to issue, and vehicle specific characteristics respectively. Under this framework, each new supported vehicle must be manually ported. During this process, the car's CAN bus communication is decoded, and is then mapped to Comma AI's structures. The culmination of this is that the internal systems only ever have to interact with the aforementioned structures, meaning the internal systems are agnostic to differences in vehicle model.

Further, the two systems feature different internal communication. Comma AI's system utilizes the messaging system, cereal, to communicate whereas CAN buses do not. In order to facilitate data exchange between the two, a microcontroller called Panda is used. Its responsibilities are twofold. For one, it acts as the communication passage between the two systems. It reads incoming CAN messages, then passes it to openpilot in a suitable format. When openpilot would like to issue a command to the car, Panda transmits the request onto the vehicle's CAN network. Panda's second responsibility revolves around safety constraints. It is there to ensure that openpilot never executes a dangerous command. For instance, it will restrict the speed at which steering commands can be issued in order to prevent needlessly jolty steering behaviour, in addition to other such commands that violate the vehicles permitted control behaviour.

Communication flows both ways; the comma device both receives information from the car and sends information to the car. The vehicle state information that it receives is used to understand how the car is moving and calculate the appropriate commands to issue. Through the CarState it is able to determine the vehicle speed, wheel speeds, and acceleration, allowing it to estimate the car's current motion. It can read the steering angle, steering rate, and driver-applied steering torque to determine the steering response required and detect driver intervention. It receives information about the pedals to determine when the driver takes control. Information from the cruise control system is used to determine whether assisted driving is engaged. Driving controls such as turn signals and gear selection inform openpilot of the driver's intentions. Radar data can be used to inform openpilot of surrounding vehicles and help maintain following distance.

After processing the vehicle's information as well as images from its own cameras, openpilot determines how the car should move. It can issue steering torque and steering angle commands to adjust the car's direction; Powertrain commands to adjust the speed; Braking commands to slow the vehicle; Cruise control commands to interact with the vehicles built in cruise control system; As well as turn signal commands to assist in automated lane changes.

Use Cases

State Diagrams

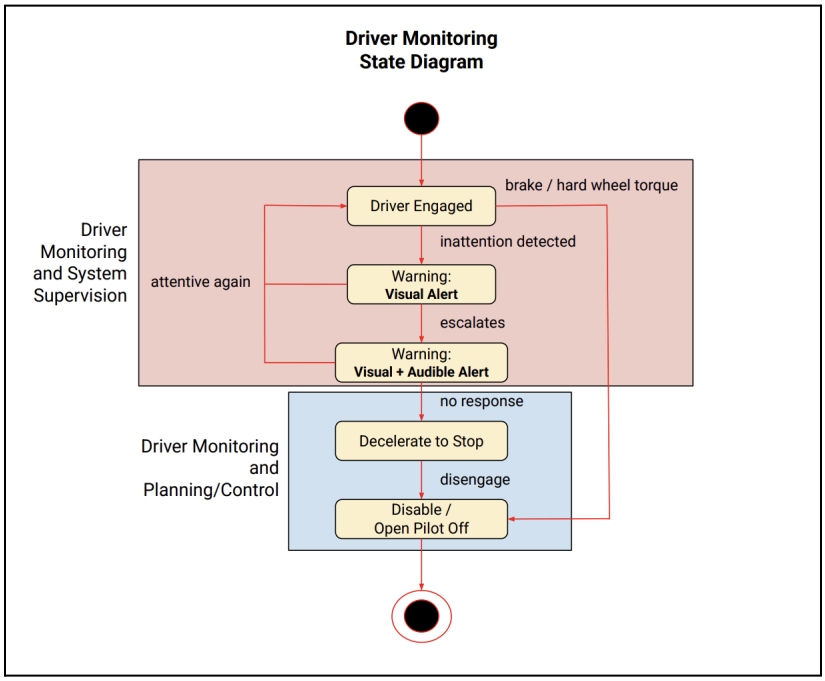


Figure 2. Supervising the driver (making sure the driver stay alert and watching the road)

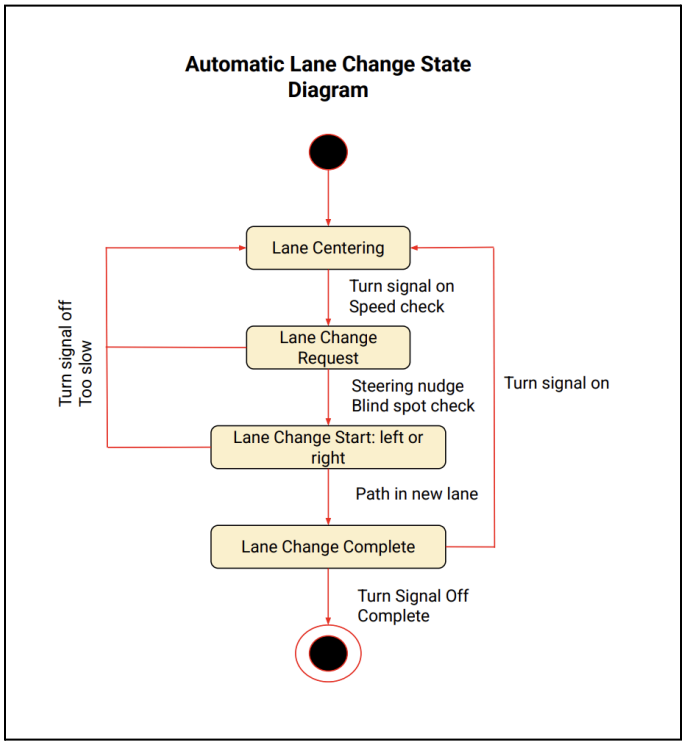


Figure 3. Changing the lane using turn signal

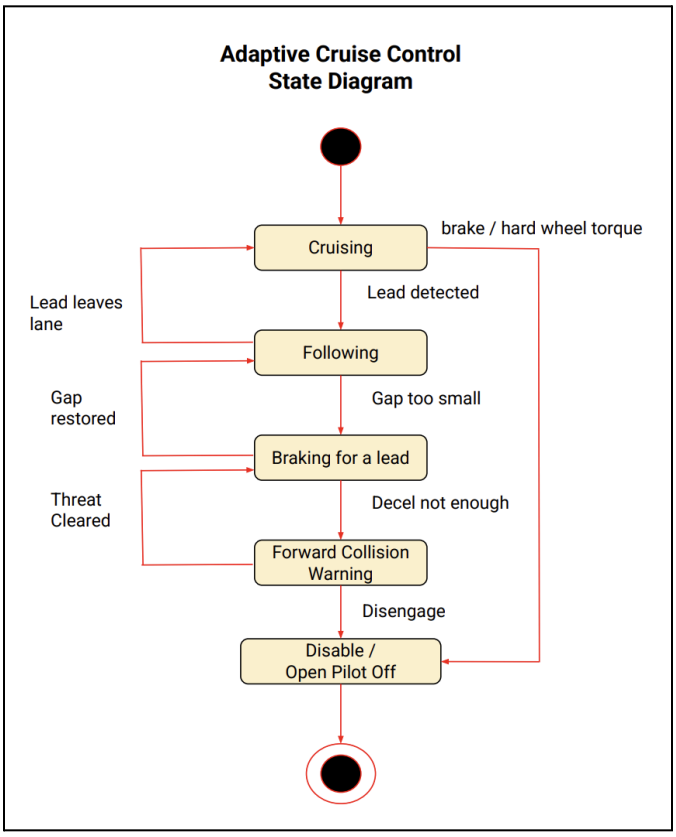


Figure 4. Keeping a safe distance from the car ahead

Sequence Diagrams

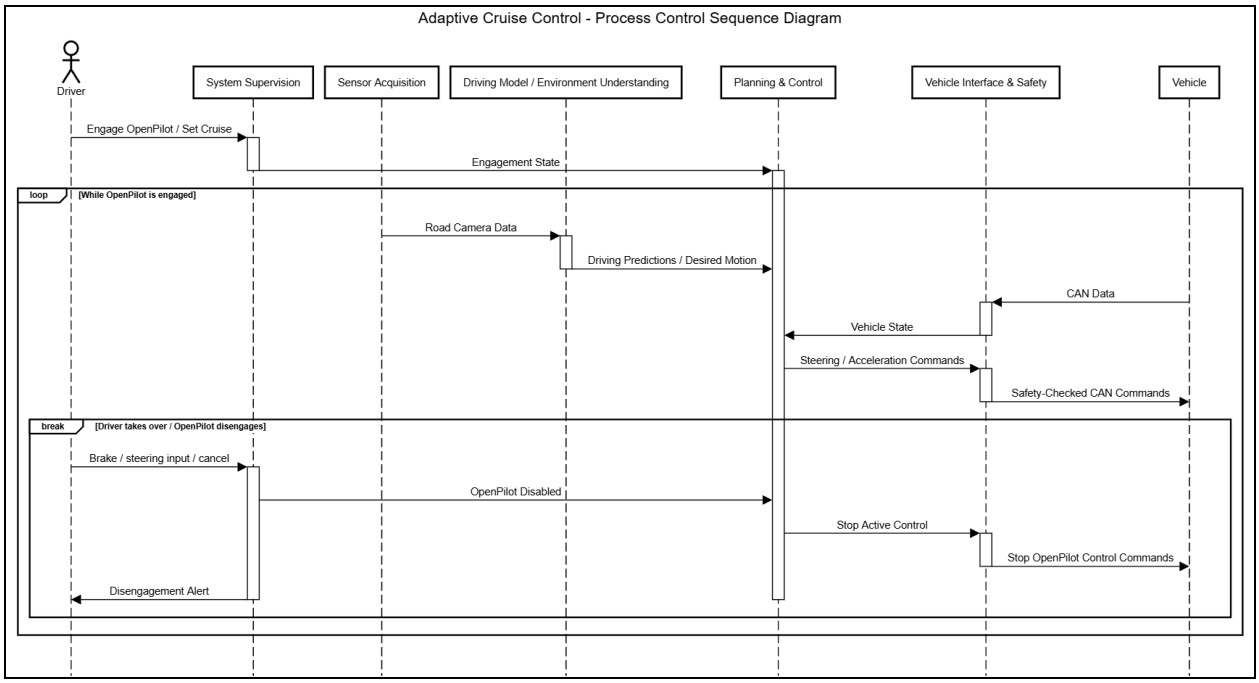


Figure 5. Sequence diagram showing subsystem interactions during Adaptive Cruise Control

Data Dictionary

Term	Type	Definition
CAN bus	Communications	Controller Area Network typically used in cars, and supports communications between components via a message based protocol
Daemon	Background program	A program that provides ongoing function without direct user control
OEM	Abbreviation	Original Equipment Manufacturer
ADAS	Abbreviation	Advanced Driver Assistance System
AGNOS	Operating System	Ubuntu based operating system used
Panda	Safety and Hardware	Vehicle and hardware safety bridge developed by comma ai
VisionIPC	Communications	Interprocess communications used by comma ai to transfer camera frames
Pipe and Filter	Architectural Style	Structure used to transform inputs and outputs into separate stages or filters
Feedback Control	Architectural Style	A closed cycle that repeatedly measures behavior and adjusts commands for best behaviors
Publish and Subscribe	Architectural Style	Publishers send information on named channels and subscribers receive without direct calls
Data repository	Architectural Style	A shared data store between components
Adapter	Pattern	A component that can translate incompatible interfaces

Naming Conventions

Term	Associated Concept / Architecture	Definition
Interprocess Communications	VisionIPC	Allows distinct operating system (OS) processes to communicate with each other, exchange data, and synchronize execution
Publishers	Pub-Sub / Event-Driven	Components that create and send data to specified

	Architecture	channels without needing to know who the subscribers are
Channels	Pub-Sub / Event-Driven Architecture	Channels (or topics) are message lines that connect publishers and subscribers
Subscribers	Pub-Sub / Event-Driven Architecture	Components that listen to specific channels to receive messages from publishers
Hertz	Pub-Sub / Event-Driven Architecture	Tells how many times per second a publisher sends data to a channel
Cereal Messages	Serialization	Cereal is short for serialization. Converts data structures and objects into a formatted stream of bytes to be stored or sent out
Logging	Observability	How the system records events, message flows, and errors over time.
Dependencies	System Architecture	External libraries, services, or processes that a system component relies on to function properly.

Lessons Learned

Throughout the course of this project, we learned several key lessons. Starting with the first lesson, we learned how to reverse engineer real-world systems at a high level. This helped us learn about how certain functions, features, and aspects of the system were connected together, and helped us understand how Openpilot actually serves its end users. However, learning this alone was not enough for us to design our own systems from scratch, which inspired us to connect what we learned about Openpilot to course material. This further revealed more architectural design decisions and hidden layers that we did not discover from our initial investigation. It also taught us how to derive the entire system into different subsystems and architectures, rather than viewing it as one monolithic system. As a result, we were able to rebuild the architectural diagrams ourselves, and significantly our understanding of the system and the course material as a whole. The last lesson we learned was how the different architectural styles and subsystems should be connected to actually serve the users. Recognizing patterns can be easy, and memorizing their connections is not an extremely difficult task either. However, actually understanding which systems should connect with each other to ensure features are working properly can be a difficult task. Simply connecting two random architectural styles or systems can cause features to fall short, or in the worst case, cause system failures that could potentially endanger users. Overall, we learned how to reverse engineer real world systems, connect our findings to course material, and learned how to effectively connect systems together to serve end users.

Conclusions

OpenPilot's architecture separates major responsibilities such as sensing, driver monitoring, planning, control, vehicle interfacing, and system support into cooperating subsystems. Its modular structure, publish-subscribe communication, feedback based control, and pipe and filter structure help the system remain extensible while supporting real time driver assistance. Overall, the architecture shows how complex vehicle functions can be organized into clear, maintainable components without relying on a single design.

References

[1] comma.ai, "What is openpilot?," openpilot Documentation. [Online]. Available: https://docs.comma.ai/. [Accessed: Oct. 23, 2026].

[2] comma.ai, "How openpilot works in 2021," comma.ai Blog, Oct. 12, 2021. [Online]. Available: https://blog.comma.ai/openpilot-in-2021/. [Accessed: Oct. 23, 2026].

[3] comma.ai, "openpilot," comma.ai. [Online]. Available: https://comma.ai/openpilot. [Accessed: Oct. 23, 2026].

[4] comma.ai, "Safety," openpilot Documentation. [Online]. Available: https://docs.comma.ai/concepts/safety/. [Accessed: Oct. 23, 2026].

[5] comma.ai, "Supported Cars," openpilot Documentation. [Online]. Available: https://docs.comma.ai/CARS/. [Accessed: Oct. 23, 2026].

[6] comma.ai, "What is a car port?," openpilot Documentation. [Online]. Available: https://docs.comma.ai/how-to/car-port/. [Accessed: Oct. 23, 2026].

[7] comma.ai, "openpilot glossary," openpilot Documentation. [Online]. Available: https://docs.comma.ai/concepts/glossary/. [Accessed: Oct. 23, 2026].

[8] comma.ai, "Logging," openpilot Documentation. [Online]. Available: https://docs.comma.ai/concepts/logs/. [Accessed: Oct. 23, 2026].

[9] comma.ai, “openpilot 0.9.7,” comma.ai Blog, Jun. 13, 2024. [Online]. Available: https://blog.comma.ai/097release/. [Accessed: Oct. 23, 2026].

[10] comma.ai, “openpilot 0.11.1,” comma.ai Blog, Jun. 5, 2026. [Online]. Available: https://blog.comma.ai/0111release/. [Accessed: Oct. 23, 2026].